Deep Reinforcement Learning

Master Summary

Stefan Eder

Based on Lecture *Deep Reinforcement Learning* SS26, JKU Linz

How to use this document. This is a structured summary built around *running text*. Explanations, derivations, and logical flow between ideas live in the prose — colored boxes are reserved for the few things that must stand out at a glance.

- **Running text** — read for understanding, derivations, and context.
- **Colored boxes** — scan for key definitions, theorems, examples, and exam notes.

Boxes lose their impact when overused. When in doubt, write prose.

Compiled March 22, 2026

Contents

1 RL Foundations **1**

1.1 What is Reinforcement Learning? 1

1.2 The Agent–Environment Interface 2

1.3 Goals and Returns 3

1.4 Policies and Value Functions 3

1.5 Exploration vs. Exploitation 5

1.6 Multi-Armed Bandits 5

1.6.1 Problem setup 5

1.6.2 Action value estimation 5

1.6.3 Regret 6

1.6.4 Algorithms 6

2 Tabular Value-Based Methods **8**

2.1 RL Taxonomy 8

2.2 Bellman Equations 8

2.3 Planning with Dynamic Programming 9

2.3.1 Policy evaluation and improvement 9

2.3.2 Policy iteration and value iteration 10

2.4 Model-Free Prediction 10

2.4.1 Monte Carlo: learn from complete episodes 10

2.4.2 Temporal-difference: learn online, one step at a time 10

2.4.3 Multi-step returns and TD(λ) 11

2.5 Model-Free Control 11

2.5.1 SARSA: on-policy TD control 11

2.5.2 Q-learning: off-policy TD control 12

2.5.3 Maximization bias and Double Q-learning 12

2.5.4 Off-policy learning and importance sampling 12

3 Value Function Approximation **13**

3.1 Learning Value Functions by Gradient Descent 13

3.2 Function Approximators 13

3.3 Action-Value Approximation and Control 14

3.4 Batch Learning and Experience Replay 14

3.5 Deep Q-Networks 15

3.6 Beyond DQN: Key Extensions 15

1

RL Foundations

Reinforcement learning (RL) is the third major paradigm of machine learning alongside supervised and unsupervised learning. In **supervised learning** we learn a mapping from labeled input–output pairs; in **unsupervised learning** we discover structure in unlabeled data. RL is different: an **agent** interacts with an **environment**, receives feedback in the form of a scalar reward signal, and learns to act so as to maximize cumulative reward. The first lecture of this course is a structured recap of the core formalism — the machinery developed here underpins every deep-RL method covered later.

1.1 What is Reinforcement Learning?

Reinforcement learning is a *computational approach of learning to make decisions from interactions*. The agent is not told what the correct action is; it must discover which actions lead to high reward by trying them. This is what separates RL from supervised learning, where target labels are provided.

Intuition & Mental Model

Five defining characteristics of RL:

- **No labeled targets** — only a scalar reward signal, not explicit supervision.
- **Delayed feedback** — reward may arrive long after the action that caused it (the *credit-assignment* problem).
- **Active data collection** — the agent's actions influence which states are visited next, so the data distribution is non-stationary and policy-dependent.
- **Sequential interactions** — decisions at time t affect states at $t+1, t+2, \ldots$, creating long-term dependencies.
- **Goal-directed** — the objective is to maximize cumulative reward, not any single step's outcome.

The overarching assumption that makes the RL framework broadly applicable is the *reward hypothesis*.

Reward Hypothesis

All goals and purposes of an agent can be described as the maximization of the *expected cumulative reward*.

This is a conjecture, not a theorem — but it is a remarkably useful design principle for defining objectives across robotics, games, language models, and beyond.

1.2 The Agent–Environment Interface

At each discrete time step $t = 0, 1, 2, \ldots$ the agent observes the current state S_t, selects an action A_t, and the environment responds with a scalar reward R_{t+1} and the next state S_{t+1}. The agent executes A_t and receives observation O_t and reward R_t; the environment receives A_t and emits O_{t+1} and R_{t+1}.

Trajectory

The agent observes state $S_t \in \mathcal{S}$ and selects action $A_t \in \mathcal{A}(S_t)$. The environment responds with reward $R_{t+1} \in \mathbb{R}$ and next state S_{t+1}. The resulting sequence

$$S_0,\ A_0,\ R_1,\ S_1,\ A_1,\ R_2,\ \ldots$$

is called a *trajectory* (or history).

A useful tool throughout RL is the conditional expectation. Recall $\mathbb{E}[X] = \sum_i x_i \Pr\{X = x_i\}$ and $\mathbb{E}[X \mid Y = y] = \sum_i x_i \Pr\{X = x_i \mid Y = y\}$. The *law of total expectation*, $\mathbb{E}[X \mid Y] = \sum_z \Pr\{Z = z \mid Y\}\, \mathbb{E}[X \mid Y, Z = z]$, is used repeatedly when expanding value functions over actions and next states.

To formalize the environment we use the Markov Decision Process framework. The slides present two equivalent definitions; the first is standard in theoretical work, the second separates transition dynamics from expected reward.

Markov Decision Process

A **Markov Decision Process (MDP)** is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma \rangle$, where

- $\mathcal{S}$ is the finite set of **states**,
- $\mathcal{A}$ is the finite set of **actions**,
- $\mathcal{P}$ specifies the **joint transition probabilities** (Markov):

$$p(s', r \mid s, a) \;=\; \Pr(S_{t+1} = s',\ R_{t+1} = r \mid S_t = s,\ A_t = a),$$

- $\gamma \in [0, 1]$ is the **discount factor**.

Alternative formulation. Separate state transitions from expected reward:

$$p(s' \mid s, a) = \sum_r p(s', r \mid s, a),$$

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_r r \sum_{s'} p(s', r \mid s, a).$$

Both formulations define the same dynamics; use whichever is more convenient.

The crucial assumption embedded in the MDP definition is the *Markov property*: the next state and reward depend on history only through the current state–action pair.

Markov Property

A state signal is *Markov* if

$$\begin{aligned}&\Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a, \text{ past})\\ &\quad = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).\end{aligned}$$

"The future is independent of the past given the present."

The state must capture all information relevant to future dynamics. In a chess game the full board position is Markov — knowing the board and the next move uniquely determines the distribution over future boards. Only tracking one piece's position would *not* be Markov, since the hidden pieces still influence legal moves.

Example: Cleaning Robot. Consider a robot vacuum with two battery states $\mathcal{S} = \{\texttt{high}, \texttt{low}\}$. From `high` it can `wait` or `search`; from `low` it can additionally `recharge` (deterministically returns to `high`, reward 0). Searching yields higher expected reward than waiting ($r_{\texttt{search}} > r_{\texttt{wait}}$) but may drain the battery, and running out of charge gives reward -3. The stochastic transition probabilities (parameterized by $\alpha, \beta \in (0, 1)$) define a small finite MDP — a concrete instance of the abstract framework above.

1.3 Goals and Returns

The agent's objective is to maximize not just the immediate reward but the long-run *return* — the cumulative discounted sum of future rewards.

Discounted Return

The **discounted return** from time t is

$$G_t \;=\; R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \;=\; \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1)$ is the **discount factor**. Discounting makes the sum finite for continuing tasks and encodes preference for sooner rewards.

Recursive identity:

$$G_t \;=\; R_{t+1} + \gamma\, G_{t+1}.$$

Starting from $G_T = 0$ and iterating backwards is far more efficient than summing forward from each t.

The discount factor γ controls the effective time horizon. As $\gamma \to 0$ only the immediate reward matters (myopic agent); as $\gamma \to 1$ distant rewards count nearly as much as immediate ones (far-sighted agent). For **episodic tasks** that terminate at time T the sum is naturally finite and $\gamma = 1$ is permissible. For **continuing tasks** with no terminal time, $\gamma < 1$ is required. A sanity check: for a continuing task with constant reward r and $\gamma < 1$, the geometric series gives $G_t = r/(1 - \gamma)$.

1.4 Policies and Value Functions

The agent's behavior is fully described by its *policy*.

Policy

A **policy** π is a mapping $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ that assigns each state–action pair the probability of selecting a in state s:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s), \qquad \sum_{a' \in \mathcal{A}} \pi(a' \mid s) = 1 \;\; \forall s \in \mathcal{S}.$$

A policy is *deterministic* if it assigns probability 1 to a single action per state. MDP policies depend only on the current state (Markov sufficiency).

The agent seeks a policy π that maximizes expected return. Two value functions are the central objects for evaluating and improving policies.

State-Value and Action-Value Functions

Given policy π, the **state-value function** and **action-value function** are

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s,\, A_t = a].$$

$v_\pi(s)$ measures how good it is to *be* in state s under π; $q_\pi(s, a)$ measures how good it is to *take action* a in s and then follow π. They are related by:

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a) = \mathbb{E}_\pi[q_\pi(S_t, A_t) \mid S_t = s].$$

Optimal policies are defined via a partial order on value functions: $\pi \geq \pi'$ iff $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$.

Optimal Value Functions

$$v_*(s) = \max_\pi v_\pi(s), \qquad q_*(s, a) = \max_\pi q_\pi(s, a).$$

The MDP is "solved" when the optimal value function is known.

Optimal Policies

For any finite MDP:

- There exists an optimal policy π^* such that $\pi^* \geq \pi$ for all π.
- All optimal policies achieve v_* and q_* simultaneously.
- There is always a *deterministic* optimal policy.

Given q_*, the optimal policy is recovered by a greedy step:

$$\pi^*(s, a) = \begin{cases} 1 & a = \arg\max_{a'} q_*(s, a'), \\ 0 & \text{otherwise.} \end{cases}$$

If multiple actions share the maximum, any (stochastic) selection among them is optimal. Crucially, knowing q_* yields π^* without needing a model of $p(s', r \mid s, a)$ — a key motivator for action-value methods.

1.5 Exploration vs. Exploitation

A learning agent must balance two competing objectives. **Exploitation**: act on current knowledge to maximize performance. **Exploration**: try actions that increase knowledge, possibly at short-term cost. Neither extreme is optimal: pure exploitation risks committing to a suboptimal action before adequate data are gathered; pure exploration never leverages what is already known.

The tension is cleanest in the *stateless* setting of multi-armed bandits, where no state transitions complicate the picture. Bandits therefore serve as the canonical sandbox for studying exploration strategies before extending them to the full MDP setting.

1.6 Multi-Armed Bandits

1.6.1 Problem setup

k-Armed Bandit

A **k-armed bandit** has k actions ("arms") $a \in \{1, \ldots, k\}$. There is a single state (non-associative setting): actions have no long-term consequences on the environment beyond the immediate reward. At each step t the agent selects arm A_t and observes reward R_t. The **true action value** is

$$q(a) \;=\; \mathbb{E}[R_t \mid A_t = a].$$

The agent maintains an estimate $Q_t(a)$ and a count $N_t(a)$. True values are unknown and must be learned from experience.

A problem is **stationary** if $q(a)$ is constant over time, and **non-stationary** if the reward distributions drift. This distinction drives the choice of estimation method.

1.6.2 Action value estimation

The natural unbiased estimator of $q(a)$ is the sample average of observed rewards:

$$Q_t(a) = \frac{\sum_{n=1}^{t} R_n \cdot \mathbb{1}_{A_n = a}}{N_t(a)}.$$

By the law of large numbers, $Q_t(a) \to q(a)$ as $N_t(a) \to \infty$. Rather than recomputing from scratch each step, the incremental form processes one observation at a time:

Incremental Update

After selecting A_t and observing R_t, update only the chosen arm's estimate:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha_t\big(R_t - Q_t(A_t)\big),$$

where $(R_t - Q_t(A_t))$ is the *prediction error*.

- $\alpha_t = 1/N_t(A_t)$: reproduces the unbiased sample average.
- **Constant** $\alpha \in (0, 1]$: exponentially recency-weighted average — older rewards count less, ideal for non-stationary problems.

All other estimates $Q_t(a)$ with $a \neq A_t$ remain unchanged.

1.6.3 Regret

Rather than tracking absolute reward, it is informative to measure how much is *lost* by not always selecting the optimal arm.

Regret

The **optimal value** is $q^* = \max_a q(a)$. The **regret** of selecting action a is

$$\Delta_a \;=\; q^* - q(a) \;\geq\; 0.$$

The **total regret** after t steps is

$$L_t \;=\; \sum_{n=1}^{t} (q^* - q(A_n)) \;=\; \sum_{n=1}^{t} \Delta_{A_n}.$$

Maximizing cumulative reward is equivalent to minimizing total regret L_t.

A good exploration strategy keeps L_t sub-linear in t — the per-step regret eventually goes to zero. Pure greedy can suffer linear regret if it commits early to a suboptimal arm.

1.6.4 Algorithms

Bandit algorithms split into two families. **Value-based** methods maintain action-value estimates $Q_t(a)$ and derive a policy from them. **Policy-based** methods maintain action preferences directly.

Greedy. $A_t \in \arg\max_a Q_t(a)$. Pure exploitation: always selects the arm with the highest current estimate. Achieves no exploration and can converge to a suboptimal arm if early samples are misleading.

ε-Greedy.

ε-Greedy Action Selection

With probability $1 - \varepsilon$ select the greedy action $A_t \in \arg\max_a Q_t(a)$. With probability ε select uniformly at random from $\{1, \ldots, k\}$.

A fixed $\varepsilon > 0$ ensures ongoing exploration but wastes a constant fraction of steps on random actions indefinitely. **GLIE** schedules (Greedy in the Limit with Infinite Exploration, e.g. $\varepsilon_k = 1/k$) decay $\varepsilon \to 0$ while still exploring infinitely often — meeting the conditions required for convergence guarantees.

Upper Confidence Bound (UCB).

UCB Action Selection

$$A_t \in \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $c > 0$ controls the exploration bonus. Each arm must be tried at least once before applying the formula.

The bonus $c\sqrt{\ln t / N_t(a)}$ is large for arms with few observations (high uncertainty) and shrinks as data accumulate. UCB explores *selectively* — prioritizing arms where evidence is weak — unlike

ε-greedy which explores uniformly regardless of confidence. If arm a is neglected, $N_t(a)$ stays constant while $\ln t$ grows, so the bonus eventually makes a worth revisiting. In short: *exploit what you are confident is good; explore what you are unsure about.*

Thompson Sampling. A Bayesian approach: maintain a *posterior distribution* over each $q(a)$ (e.g. a Beta distribution for Bernoulli rewards). At each step, draw one sample from each arm's posterior and select the arm with the highest sample. Arms with high posterior variance are sampled frequently; as evidence accumulates the posterior concentrates around the true value and exploitation dominates. Thompson Sampling achieves asymptotically optimal regret and often outperforms UCB in practice.

Gradient Bandits (policy-based).

Preference Parameters

Instead of estimating action values, gradient bandits maintain a real-valued **preference** $H_t(a) \in \mathbb{R}$ for each arm. Only *differences* matter: adding a constant to all preferences leaves the policy unchanged.

Preferences are converted to a sampling distribution via the softmax:

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))}.$$

After observing reward R_t with running baseline $\bar{R}_t$ (often the running average reward), preferences are updated by stochastic gradient ascent on expected reward:

$$H_{t+1}(a) = \begin{cases} H_t(a) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(a)) & a = A_t, \\ H_t(a) - \alpha(R_t - \bar{R}_t)\,\pi_t(a) & a \neq A_t. \end{cases}$$

What the Gradient Update Does

Think of π_t as a weighted lottery. If $R_t > \bar{R}_t$ (better than average): increase $H(A_t)$, making the selected arm more likely; slightly decrease the rest. If $R_t < \bar{R}_t$: decrease $H(A_t)$ and increase the rest. The factors $(1 - \pi_t(A_t))$ and $\pi_t(a)$ scale the update: a surprising action (low π_t) triggers a large adjustment; a near-certain action changes little. Unlike value-based methods, gradient bandits have no explicit exploration rate — exploration emerges from the softmax sampling itself.

2

Tabular Value-Based Methods

Lecture 02 is organized around a single central question: *how do we actually solve for an optimal policy?* The answer depends on two things — whether we have access to a model of the environment, and whether we compute expectations exactly or estimate them from samples. Those two choices carve out the three algorithm families covered here: Dynamic Programming, Monte Carlo, and Temporal-Difference learning.

2.1 RL Taxonomy

Before diving in, it helps to know where different algorithms sit in the broader landscape. RL algorithms can be characterized along three orthogonal axes.

The first axis is **what gets learned**. *Value-based* methods maintain an explicit value function and derive the policy implicitly (e.g. take the action with the highest Q-value). *Policy-based* methods directly parameterize and optimize the policy without an intermediate value function. *Actor-critic* methods maintain both: a policy (actor) that decides what to do, and a value function (critic) that evaluates how good the current situation is.

The second axis is **which data are used**. *On-policy* methods learn about the same policy that generates the data — evaluation and behavior are the same policy. *Off-policy* methods separate these two roles: the *target policy* π is what we want to learn about, while the *behavior policy* μ generates the actual data. This separation is powerful: it allows reusing data from old policies, learning from human demonstrations, or learning a greedy policy while behaving exploratorily. *Offline* methods go even further and learn from a fixed pre-collected dataset with no environment interaction at all.

The third axis is **whether a model is available**. *Model-based* methods use or learn a model of the environment dynamics $p(s', r \mid s, a)$ and can plan ahead by simulating transitions. *Model-free* methods estimate value functions or policies directly from experience, never consulting a model.

These three axes are independent — DQN is value-based, off-policy, and model-free; AlphaZero is actor-critic, on-policy, and model-based.

2.2 Bellman Equations

The Bellman equations are the mathematical backbone of tabular RL. They express a simple recursive insight: the value of being in a state equals the immediate reward you expect plus the discounted value of where you end up.

Bellman Expectation and Optimality Equations

For any policy π, the value functions satisfy the *expectation equations*:

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_\pi(s') \right],$$

$$q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) \sum_{a'} \pi(a' \mid s')\, q_\pi(s',a').$$

The *optimal* value functions satisfy the *optimality equations*:

$$v_*(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_*(s') \right],$$

$$q_*(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) \max_{a'} q_*(s',a').$$

The expectation equations are *linear* in the value function — in principle, you could solve them as a linear system in $O(|\mathcal{S}|^3)$ operations. For anything but tiny MDPs, that is computationally prohibitive, so iterative methods are needed.

The optimality equations look similar but replace the policy-weighted average with a max. That max makes them nonlinear, so there is no closed-form solution. The key practical insight is this: once you have q_*, the optimal policy is free — just be greedy, $\pi^*(s) = \arg\max_a q_*(s,a)$. Using v_* instead requires the model to compute the arg max, whereas q_* gives you the optimal action directly without any model.

2.3 Planning with Dynamic Programming

Dynamic programming computes optimal policies *given a perfect model* of the environment. DP uses the model to form exact expected-value backups rather than sampling. Every DP algorithm alternates between two steps: **policy evaluation** (compute how good the current policy is) and **policy improvement** (make the policy better).

2.3.1 Policy evaluation and improvement

Given a fixed policy π, we want to compute its value function v_π exactly. The trick is to treat the Bellman expectation equation not as a system of equations to solve, but as an *update rule*. Starting from any initialization V_0, iterating the update

$$V_{k+1}(s) \leftarrow \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V_k(s') \right]$$

for all states s simultaneously converges to v_π. In practice we stop when the maximum change across states falls below a small threshold ε.

Once we have $V \approx v_\pi$, policy improvement is straightforward. The **policy improvement theorem** guarantees that choosing the greedy action at every state — $\pi'(s) = \arg\max_a q_\pi(s,a)$ — always yields a policy at least as good as π. If $v_{\pi'}(s) = v_\pi(s)$ for all s, the policy was already optimal.

2.3.2 Policy iteration and value iteration

Policy iteration repeats evaluation and improvement in alternation: run iterative policy evaluation to convergence, extract the greedy policy, repeat. It is guaranteed to reach the optimal policy after a finite number of improvements, but running evaluation fully to convergence on each iteration is expensive.

Value iteration is a smarter shortcut: do not wait for evaluation to converge. Instead, apply a *single* backup using the optimality operator — taking the max rather than the policy-weighted sum:

$$V_{k+1}(s) \;\leftarrow\; \max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V_k(s') \right].$$

This is equivalent to interleaving one step of evaluation with one step of improvement. Value iteration converges asymptotically (no finite termination guarantee), but is typically more efficient per iteration than running full policy evaluation.

Limitation of DP

Both algorithms require knowing $p(s', r \mid s, a)$ for every transition. In most real-world settings this model is unavailable. Model-free methods, covered next, replace the model-based expectation with samples drawn from the environment.

2.4 Model-Free Prediction

The prediction task is: given a policy π, estimate v_π from experience alone, without any model. Two fundamentally different approaches exist.

2.4.1 Monte Carlo: learn from complete episodes

Monte Carlo (MC) methods estimate $v_\pi(s)$ by averaging the actual returns observed when starting from s. Since $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and G_t is observable at episode end, we can use the sample average as the estimator, updated incrementally:

$$V(S_t) \;\leftarrow\; V(S_t) + \alpha\big[G_t - V(S_t)\big].$$

G_t is an *unbiased* estimate of $v_\pi(S_t)$ — on average it equals the true value. The downside is variance: G_t is a sum of many random rewards, so individual estimates can vary wildly, and the method requires waiting until the episode ends.

2.4.2 Temporal-difference: learn online, one step at a time

TD methods trade unbiasedness for immediacy. Instead of waiting for the actual return G_t, TD uses the Bellman equation to construct an estimate after just one step — the **TD(0) target** $R_{t+1} + \gamma V(S_{t+1})$. The update is:

TD(0) Update Rule

$$V(S_t) \;\leftarrow\; V(S_t) + \alpha \underbrace{\big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big]}_{\text{TD error } \delta_t}.$$

TD(0) is model-free like MC, bootstraps like DP, and can update after *every step* without waiting for episode end.

The TD target is *biased* because $V(S_{t+1})$ is only an approximation — we are plugging an estimate back into its own update. But it has far lower variance than G_t, because only a single reward is random rather than a whole trajectory's worth. In practice, TD often learns faster even though it introduces this bias.

Information propagation differs too. In MC, a surprising outcome at the end of an episode immediately updates all states visited along the way. In TD, that information trickles backward only one step at a time.

2.4.3 Multi-step returns and TD(λ)

n-step TD provides a smooth interpolation between TD(0) ($n = 1$) and MC ($n = \infty$) by collecting n actual rewards before bootstrapping:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Rather than committing to one n, TD(λ) mixes *all* n-step returns simultaneously using geometrically decaying weights:

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t.$$

Setting $\lambda = 0$ recovers TD(0); $\lambda = 1$ recovers MC. Intermediate values get the best of both worlds: lower variance than MC while propagating information faster than one-step TD. In practice the λ-return is implemented online via **eligibility traces**, which maintain a decaying memory of recently visited states and spread the TD error backward proportionally.

2.5 Model-Free Control

For control, we need action-value estimates $Q(s, a)$ rather than state values. The reason is simple: to be greedy over v_π, you need to compute $\arg\max_a \big[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V(s')\big]$, which requires a model. Being greedy over $Q(s, a)$ is model-free: just pick $\arg\max_a Q(s, a)$.

The general framework is **generalized policy iteration**: alternate between approximate policy evaluation (using MC or TD to estimate $Q \approx q_\pi$) and policy improvement (act greedily with respect to Q). The catch is exploration: a purely greedy policy never tries suboptimal actions, so $Q(s, a)$ for those actions never gets updated and the policy can get stuck. The standard fix is ε-greedy: with probability ε take a random action, otherwise act greedily. To guarantee convergence to the optimal policy, we need the exploration schedule to satisfy GLIE — *Greedy in the Limit with Infinite Exploration*: every state-action pair must be visited infinitely often, yet the policy must eventually converge to greedy. An ε-greedy schedule with $\varepsilon_t \to 0$ and $\sum_t \varepsilon_t = \infty$ satisfies GLIE.

2.5.1 SARSA: on-policy TD control

SARSA applies the TD idea directly to action values. At each step we observe the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, then sample the next action A_{t+1} from the current policy and update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big].$$

The five quantities $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ give the algorithm its name. SARSA is *on-policy*: the action A_{t+1} used in the target is actually sampled from the current ε-greedy policy, so the estimate converges to q_π for the policy being followed.

A variant, **Expected SARSA**, replaces the sampled A_{t+1} with an expectation over the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

This reduces variance because it averages over the randomness in A_{t+1} rather than sampling it.

2.5.2 Q-learning: off-policy TD control

Q-learning breaks the link between behavior and learning target. Regardless of which action the agent *actually* takes next, the update uses the *greedy* action at the next state:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

This is a sample-based approximation of the Bellman optimality equation for q_*, and under standard conditions Q converges to q_* regardless of the behavior policy (which just needs to ensure all state-action pairs get visited). No importance-sampling correction is needed because the max target does not depend on the behavior policy at all.

In one sentence: SARSA evaluates what the agent *actually does*; Q-learning evaluates what the agent *would do if it were greedy*.

2.5.3 Maximization bias and Double Q-learning

There is a subtle problem with the max in Q-learning. When Q values are estimated with noise (as they always are from finite data), taking the maximum over noisy estimates produces a value that is systematically *too high* — the noise pushes the max upward. This is called **maximization bias**.

Double Q-Learning

Maintain two independent estimates Q^A and Q^B. Use one to *select* the action and the other to *evaluate* it:

$$Q^A(S_t, A_t) \leftarrow Q^A(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q^B \left(S_{t+1}, \arg\max_a Q^A(S_{t+1}, a) \right) - Q^A(S_t, A_t) \right].$$

At each step, randomly choose which estimate to update; the other plays the evaluation role. Decoupling selection from evaluation breaks the positive correlation that drives the bias upward.

2.5.4 Off-policy learning and importance sampling

When using MC or SARSA-style updates off-policy — learning about π from data generated by μ — the mismatch in action probabilities must be corrected. The **importance sampling ratio** $\rho_t = \pi(A_t \mid S_t)/\mu(A_t \mid S_t)$ reweights each transition to look as if it came from π. For full trajectories the ratio is a product over the whole path. Off-policy MC multiplies the return by the full trajectory ratio ρ_t^T; off-policy SARSA uses only a single-step ratio ρ_{t+1}. Q-learning sidesteps the whole issue because its max target is already policy-agnostic.

3

Value Function Approximation

Every tabular method we have seen so far stores a separate number for each state or state-action pair. That is fine for toy problems, but it breaks at scale. Backgammon has around 10^{20} states; Go has around 10^{170}; robotic manipulation lives in a continuous space with infinitely many states. No table can hold all of those, and no algorithm could visit each state enough times to learn a reliable estimate.

The solution is to give up on exact representation and *approximate* the value function with a parameterized function $v_{\mathbf{w}}(s) \approx v_\pi(s)$, where $\mathbf{w}$ is a weight vector with far fewer entries than there are states. The approximation generalizes: updating $\mathbf{w}$ from one state implicitly updates the estimated values of nearby states that share features. When neural networks play the role of this approximator, the whole framework becomes **deep reinforcement learning**.

3.1 Learning Value Functions by Gradient Descent

The natural objective is to minimize the mean-squared difference between the approximate and the true value function over states visited under π:

$$J(\mathbf{w}) = \mathbb{E}_\pi\left[\left(v_\pi(S) - v_{\mathbf{w}}(S)\right)^2\right].$$

Stochastic gradient descent gives the parameter update at each visited state S:

$$\Delta\mathbf{w} = \alpha\big(v_\pi(S) - v_{\mathbf{w}}(S)\big)\nabla_{\mathbf{w}} v_{\mathbf{w}}(S).$$

The problem is that $v_\pi(S)$ is unknown — we can never query the oracle. So we substitute it with a **target** derived from observed data. The MC target is the actual return G_t (unbiased but noisy). The TD(0) target is the one-step bootstrapped estimate $R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$ (lower variance, but biased because it depends on $\mathbf{w}$ itself). The TD(λ) target G_t^λ sits between the two extremes.

This dependency is important: the TD target changes as $\mathbf{w}$ updates, so we are not performing true gradient descent — we are ignoring the gradient through the target. This is called a **semi-gradient** method. It works well empirically, but convergence guarantees are weaker than for full gradient descent, and there are regimes where it can diverge (see the Deadly Triad, below).

3.2 Function Approximators

Linear approximation represents each state by a feature vector $\mathbf{x}(s) \in \mathbb{R}^d$ and computes the value as a dot product: $v_{\mathbf{w}}(s) = \mathbf{x}(s)^\top \mathbf{w}$. The squared-error objective is then quadratic in $\mathbf{w}$,

guaranteeing a unique global minimum, and the SGD update simplifies to $\Delta\mathbf{w} = \alpha(\text{target} - v_{\mathbf{w}}(S))\mathbf{x}(S)$. Tabular lookup is actually a special case of linear approximation: represent each state with a one-hot vector, and the weights $\mathbf{w}$ directly encode the value table. The difference is that one-hot features provide no generalization between states, whereas a shared feature representation does.

Challenges Specific to RL

Unlike supervised learning, function approximation in RL faces three compounding difficulties: **non-i.i.d. data** (consecutive transitions are strongly correlated, not independent samples); **policy-dependent data distribution** (the states the agent visits change as the policy improves); and **non-stationary targets** (in TD methods, the regression target changes every time $\mathbf{w}$ is updated, because the target network depends on the same weights being trained). Together these can destabilize learning, especially with nonlinear approximators.

In practice, the choice of approximator involves a three-way trade-off. *Tabular* methods have the best theory but cannot scale. *Linear* methods have reasonably strong theory but require hand-crafted features. *Nonlinear* methods — especially deep neural networks — lack tight theoretical guarantees but scale well and eliminate the need to design features by hand. Neural networks are the dominant choice in modern deep RL.

3.3 Action-Value Approximation and Control

For control we approximate $q_\pi(s, a)$ rather than $v_\pi(s)$, because the greedy improvement step $\pi'(s) = \arg\max_a q_{\mathbf{w}}(s, a)$ then requires no model. There are two natural network architectures. *Action-in* takes (s, a) as input and produces a single Q-value — you need one forward pass per action to find the best one. *Action-out* takes s as input and outputs a Q-value for every action simultaneously — much more efficient when the action space is discrete.

The same MC / TD(0) / TD(λ) targets used for state values apply directly to action values: the oracle $q_\pi(S, A)$ is replaced by G_t, by $R_{t+1} + \gamma q_{\mathbf{w}}(S_{t+1}, A_{t+1})$, or by G_t^λ. The control loop then mirrors GPI: approximate policy evaluation followed by ε-greedy improvement.

The Deadly Triad

Combining **function approximation**, **bootstrapping** (TD-style targets), and **off-policy learning** in the same algorithm can cause divergence. Each ingredient alone is fine; together they can form a feedback loop where weights drift to infinity. MC avoids it by not bootstrapping; on-policy TD with linear approximation avoids it by restricting the data distribution. Non-linear, off-policy TD — the core of DQN-style algorithms — sits right inside the danger zone, which is why DQN needs special stabilization tricks.

3.4 Batch Learning and Experience Replay

Plain online SGD is sample-inefficient: each transition is seen once and then discarded. Worse, consecutive transitions are highly correlated, which makes the gradient estimates noisy and can cause oscillations.

Experience replay addresses both problems. The agent stores all transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ in a **replay buffer** $\mathcal{D}$. At each training step, a random mini-batch is sampled from $\mathcal{D}$ and used

to compute a gradient update:

$$\Delta \mathbf{w} = \alpha(r + \gamma v_{\mathbf{w}}(s') - v_{\mathbf{w}}(s))\nabla_{\mathbf{w}} v_{\mathbf{w}}(s).$$

Random sampling breaks the temporal correlations. Each transition can be replayed many times, improving sample efficiency. The buffer also acts as a stabilizing data distribution: because it contains transitions from many past policies, the training distribution changes slowly even as the current policy evolves quickly.

3.5 Deep Q-Networks

Applying Q-learning directly with a neural network runs into trouble. Consecutive transitions are correlated (SGD assumes i.i.d. data). The TD target $r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a')$ depends on $\mathbf{w}$, so every weight update shifts the target — like trying to hit a moving goalpost. And the combination of off-policy data, function approximation, and bootstrapping puts us squarely in the deadly triad.

DQN (Mnih et al., 2015) tames all three with two techniques. **Experience replay** (described above) breaks correlations and enables data reuse. **Fixed target networks** address the moving-goalpost problem: maintain a second, frozen network $Q_{\mathbf{w}^-}$ and compute targets from it, $y = r + \gamma \max_{a'} Q_{\mathbf{w}^-}(s', a')$, only periodically copying $\mathbf{w}$ into $\mathbf{w}^-$. Between copies the targets are stable, making training resemble supervised learning.

DQN Training Loop

For each step: collect $(s_t, a_t, r_{t+1}, s_{t+1})$ with ε-greedy and store in $\mathcal{D}$. Sample a mini-batch $\{(s_j, a_j, r_j, s'_j)\}_{j=1}^{B}$. Compute targets $y_j = r_j + \gamma \max_{a'} Q_{\mathbf{w}^-}(s'_j, a')$ (or $y_j = r_j$ if terminal). Minimize $L(\mathbf{w}) = \frac{1}{B}\sum_j (y_j - Q_{\mathbf{w}}(s_j, a_j))^2$ w.r.t. $\mathbf{w}$ only (target y_j treated as a constant). Every C steps: $\mathbf{w}^- \leftarrow \tau\,\mathbf{w} + (1-\tau)\,\mathbf{w}^-$.

DQN was evaluated on 49 Atari 2600 games with *the same architecture and hyperparameters* across all games. Raw pixels were preprocessed to 84×84 grayscale; four frames were stacked to give the network a sense of motion; a CNN processed these into Q-values for each action. DQN outperformed all prior RL methods on 43 of 49 games and reached human-level performance on 29 of them — the first demonstration of general-purpose deep RL at scale.

3.6 Beyond DQN: Key Extensions

In the years after DQN, six targeted improvements addressed its remaining weaknesses. **Rainbow** (Hessel et al., 2018) combines all six into one agent, reaching DQN's final performance in 7× fewer environment steps.

Double DQN.. DQN's target uses the same frozen network to both *select* the best action and *evaluate* it — the same maximization bias problem as tabular Q-learning. Double DQN decouples these: the online network selects, $a^* = \arg\max_{a'} Q_{\mathbf{w}}(s', a')$, and the target network evaluates, $y = r + \gamma Q_{\mathbf{w}^-}(s', a^*)$.

Prioritized experience replay.. Uniform replay gives equal attention to all transitions, but not all are equally useful. Transitions with large TD error $|\delta_i|$ represent situations the network still gets wrong, so they should be replayed more often. Prioritized replay samples transition i with probability proportional to $|\delta_i|^\alpha / \sum_k |\delta_k|^\alpha$, and corrects the resulting distributional bias with importance-sampling weights.

Dueling networks.. Every Q-value can be decomposed into two parts: how good is this *state*, and how much *better* is this action relative to the average? Formally, $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$. A dueling network has a shared convolutional backbone that feeds into two separate heads (value and advantage). This is particularly useful when many actions have similar value: the value stream can still be trained from every transition, even when a specific action was not taken.

Multi-step returns.. Standard DQN uses a 1-step TD target. Replacing it with an n-step return $y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \max_{a'} Q_{\mathbf{w}^-}(S_{t+n}, a')$ propagates reward signals faster through the network. Intermediate values like $n = 3$ often work best.

Distributional RL.. Standard Q-learning estimates $\mathbb{E}[G_t]$, but much information is lost by collapsing the return to a scalar. Distributional RL instead models the full *distribution* of returns $Z(s,a)$, satisfying the distributional Bellman equation $Z(s,a) \stackrel{d}{=} R + \gamma Z(s',a')$. Training with cross-entropy loss on the return distribution provides a richer learning signal and leads to better representations.

Noisy Networks.. ε-greedy exploration adds the same random behavior in every state, ignoring the agent's uncertainty. Noisy Networks replace deterministic linear layers with stochastic ones: $y = (\mu^w + \sigma^w \odot \varepsilon^w)x + (\mu^b + \sigma^b \odot \varepsilon^b)$, where the noise parameters σ^w, σ^b are *learned*. The network automatically reduces noise in states where it is confident and maintains higher noise where it is uncertain, providing state-dependent exploration without a hand-tuned ε.